

Ethan Tran

ethantran@quwin.dev | linkedin.com/in/kien-ethan-tran | github.com/quwin

EDUCATION

California State University Sacramento
Bachelor of Science, Computer Science

Expected December 2027
GPA: 3.47

TECHNICAL SKILLS

Languages: Rust, C++, Python, Go, SQL

Databases & Data Systems: SQLite, PostgreSQL, MySQL, MongoDB, Redis, Qdrant, Apache DataFusion, Arrow, Parquet

Distributed Systems & Performance: replication, concurrency, multi-threading, CUDA, profiling, vectorization

Cloud & Infrastructure: Amazon Web Services (AWS), Google Cloud Platform (GCP), Linux, Docker, Git, GitHub Actions

WORK EXPERIENCE

Software Engineer Intern

July 2026 - Present

Perio Specialist

- Architected a database system with application-level replication across networked workstations to track dental implant inventory.
- Implemented transactional inventory updates, idempotent writes, and optimistic concurrency checks to prevent duplicate stock changes and detect conflicting edits.
- Designed persistent offline queues and automated backups to support outage-tolerant data capture and database recovery.
- Built a pre-processing pipeline that standardized patient radiographs into model-ready features to streamline dataset preparation.
- Developed landmark post-processing to align predictions with tooth boundaries, reducing mean localization error by 4% on annotated dental radiographs.

AI Engineering Extern

June 2026 - July 2026

Pfizer-hosted Externship Program

- Built a Python document-processing pipeline using OCR to extract, classify, and route content from 1,000+ page pharmaceutical files.
- Implemented configurable dense and sparse embedding indexes in a Qdrant vector database, supporting flexible query execution.
- Benchmarked extraction, retrieval, and routing across segmentation and indexing changes, improving retrieval accuracy by 14%.
- Deployed typed API endpoints returning source-backed results with validity, confidence, and completeness signals for downstream document-processing workflows.
- Produced technical documentation comparing pipeline trade-offs and deployment recommendations, helping stakeholders assess production rollout readiness.

OPEN SOURCE CONTRIBUTIONS

Apache SQL Query Engine - apache / datafusion

Rust, SQL

- Reviewed and proposed a fix for a query optimization issue causing redundant per-row traversal for reused computations.
- Identified and helped resolve an execution issue ignoring DELETE predicates and removing unintended rows.
- Improved SQL validation by rejecting invalid user-defined function placeholders at definition time instead of runtime.

Datadog Observability Data Pipeline - vectordev / vector

Rust, Lua

- Increased Lua transform reliability by replacing unimplemented metric tag handling with structured errors.

Freebuff Coding Agent Harness - CodebuffAI / freebuff

TypeScript

- Fixed MCP tool argument loss by preserving complex schemas through model registration while maintaining runtime validation.

PROJECTS

Distributed API Gateway + Rate Limiter

Go, Lua, Redis, Nginx, Docker, Prometheus, AWS

- Developed a Go API gateway combining API-key authentication, reverse-proxy routing, quota headers, and per-principal rate limiting.
- Supported fixed-window, token-bucket, and sliding-window policies through interchangeable in-memory and Redis-backed limiter implementations.
- Replaced client-side read-modify-write sequences with atomic Lua scripts to prevent races in shared limiter state.
- Configured Nginx across three Go gateway instances for local multi-replica validation.
- Validated limiter correctness with deterministic tests covering burst refills, expiration boundaries, cancellation, and Redis state transitions.

GPU-Accelerated Postflop Poker Solver

C++, CUDA, FLTK

- Developed and deployed a custom No-Limit Hold'em postflop solver using Counterfactual Regret Minimization (CFR) across configurable game trees.
- Engineered flattened memory layouts and GPU kernels to maximize parallel execution, increasing iteration throughput by 7%.
- Applied memory-aware batching to support large workloads on consumer GPUs, reducing memory requirements by up to 60%.
- Profiled compute, transfer, and allocation bottlenecks, restructuring memory access and execution to improve cache locality and reduce synchronization overhead.